ACKNOWLEDGEMENT

I wish to express my sincerest appreciation to all those who have helped in making the completion of the Expense Tracker project possible. In particular, I would like to express my gratitude to my project guide for his valuable assistance, guidance, and constructive suggestions that he offered during the development of the project. I would like to convey my sincere gratitude to the faculties and instructors of the BSc CSIT program who have given me the necessary skills and knowledge to accomplish this project. My sincere thanks go to my friends and fellow students for their encouragement and technical support that they extended while working on the project. I appreciate the efforts of the open-source communities and the people behind React.js, Node.js, Express.js, MongoDB, JWT, and bcrypt who made it possible for me to develop the project. My sincerest thanks go to my parents and my family members for their encouragement and support during my college life.

ABSTRACT

Expense Tracker is an online application created using MERN Stack (MongoDB, Express.js, React.js, and Node.js). It allows users to manage their personal expenses effectively. The system creates a protected environment that will enable users to register themselves, login and conduct basic operations related to the management of personal expenses including creation, viewing, modification, and deletion of expense records. The system uses JSON Web Tokens for the purpose of user authentication while encryption of passwords is done using bcrypt. Personal expense records are stored independently in MongoDB, and no other user except the registered one can view his/her record. The system is created in such a way that it allows categorization of expenses and monitoring of personal spending in an easy manner using an intuitive interface. RESTful API ensures seamless communication between front-end and back-end.

Keywords: *Expense Tracker, MERN Stack, Personal Finance, Expense Management, React.js, Node.js, Express.js, MongoDB, JSON Web Token (JWT), RESTful API.*

Table of Contents

List of Figures

List of Tables

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JWT	JSON Web Token
MERN	MongoDB, Express.js, React.js, Node.js
MongoDB	MongoDB Database
REST	Representational State Transfer
UI	User Interface
URL	Uniform Resource Locator
JSON	JavaScript Object Notation
IDE	Integrated Development Environment
VS Code	Visual Studio Code
npm	Node Package Manager
DBMS	Database Management System

1. INTRODUCTION

1.1. Introduction

Budgeting and management of personal expenditure is one of the most important processes for financial stability and making budgeting decisions. But there are a lot of people who continue using traditional ways of recording daily expenditures like writing notes down or using spreadsheets, which are time-consuming and complicated to use. In order to make things easier, modern technology helps people build digital tools for monitoring financial transactions. One of them is a web app called Expense Tracker built using MERN Stack (MongoDB, Express.js, React.js, and Node.js), which allows users to record, manage, and control their personal expenses in a secure way.

1.2. Problem Statement

It is very challenging for many people to be able to manage their expenses on a daily basis because of the use of manual means of record keeping or software that is either too complicated or does not have enough security measures. There are existing systems that might offer some additional features which are not necessary or an easy-to-use interface to manage one's expenses. In order to solve this problem, the suggested Expense Tracker can be used.

1.3. Objectives

A need for developing a web-based Expense Tracker application using MERN Stack that will allow the user to conveniently keep track of his/her expenses by means of an easy and interactive interface.

1.4. Scope and Limitations

1.4.1. Scope

- Allows users to add, view, update, and delete personal expense records.
- Offers a responsive web interface accessible from desktop and mobile browsers.

Limitations

- The system currently manages only expenses and does not include income tracking or budgeting features.

1.5. Development Methodology

The Expense Tracker application was built utilizing the Agile software development methodology. This process involved different iterative steps that include requirement analysis, system design, implementation, testing, and evaluation. The backend API, the front-end GUI, and the database were developed and tested iteratively to guarantee reliability and maintenance.

1.6. Report Organization

- **Chapter 1** introduces the project, including the problem statement, objectives, scope, limitations, development methodology, and report organization.
- **Chapter 2** presents the background study and literature review related to expense management systems and the technologies used.
- **Chapter 3** discusses the system analysis, including requirements, feasibility analysis, and system models.
- **Chapter 4** describes the system design, including database design, interface design, and deployment architecture.
- **Chapter 5** explains the implementation process, development tools, and testing performed to validate the system.
- **Chapter 6** concludes the project and provides expected outcomes along with recommendations for future improvements.

2. BACKGROUND STUDY AND LITERATURE REVIEW

2.1. Background Study

Expense Tracker is a web application which will assist people in managing their own expenses through a digital platform. Managing of one's expenses manually, by means of notebooks or spreadsheets, might be complicated and may lead to wrong information. Modern technologies make it possible to create a convenient and secure system for keeping of financial records.

The system is built upon the stack called MERN Stack, which includes MongoDB, Express.js, React.js, and Node.js. The client-server architecture allows the React.js based frontend to work in conjunction with backend via the RESTful API.

One of the core concepts utilized in the project is user authentication and authorization. JSON Web Tokens are used for creation of secure user sessions so that users may keep their private information secret and access it only when authorized. Passwords of users are hashed in order to avoid any data leaks.

A database like MongoDB has been chosen due to its versatility and capacity to store structured expense records. It should be noted that the application also follows security measures in web development such as securing API routes, using environment configuration, and input validation.

In conclusion, it can be stated that the Expense Tracker utilizes the latest technologies of full-stack development, secure authentication, and databases.

2.2. Literature Review

Traditional personal expense tracking is commonly performed using manual methods such as notebooks, receipts, and spreadsheet applications. Although these methods are simple, they are inefficient for maintaining large amounts of financial information and may lead to calculation errors and loss of records[1].

With the growth of digital technologies, various expense management applications have been developed to automate financial tracking. These applications provide features such as

expense categorization, transaction history, and financial summaries, allowing users to monitor their spending habits more effectively[2].

Research on personal finance management systems highlights the importance of security and privacy because financial information is sensitive. Secure authentication methods such as JWT-based authorization and password hashing techniques like bcrypt are widely used in modern web applications to protect user data[3].

Studies also show that NoSQL databases such as MongoDB are suitable for applications that require flexible data structures. Expense records may contain different categories, descriptions, and payment details, making MongoDB a suitable choice due to its schema flexibility and scalability[4].

Modern web application development increasingly uses frameworks such as React, Node.js, and Express.js because they support component-based development, efficient API communication, and scalable system architecture. The combination of these technologies in the MERN Stack provides a reliable approach for developing secure and user-friendly expense management applications[5].

3. SYSTEM ANALYSIS

3.1. System Analysis

Systems analysis includes the determination of the requirements, feasibility, and functionality of the intended system. The system expense tracker is designed to offer a safe and reliable means through which individuals can manage their expenses using the digital medium. System analysis will aim at determining the needs of the users, the requirements of the system, and its feasibility.

3.1.1. Requirement Analysis

Requirement analysis determines the functions and constraints of the system. The requirements of the Expense Tracker are divided into functional and non-functional requirements.

3.1.2. Functional Requirements

- Users can register and log in securely using email and password authentication.

- The system authenticates users using JWT-based authorization.
- Users can add new expense records with details such as title, amount, category, payment method, and date.
- Users can view their expense history and manage existing records.
- Users can update or delete their personal expense records.
- The system ensures that each user can access only their own expenses.
- The application stores user and expense information securely in MongoDB.

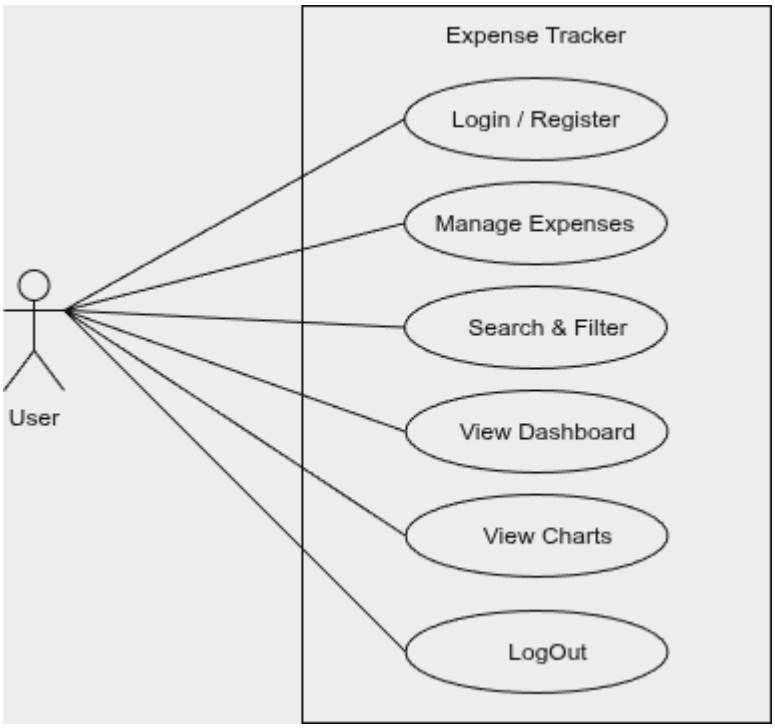

Figure 3.1: Use Case Diagram of Expense Tracker

3.1.3. Non-Functional Requirements

- **Scalability:** The system can support increasing numbers of users and expense records using MongoDB's flexible database structure.
- **Reliability:** The application maintains accurate expense records and provides consistent performance.
- **Security:** JWT authentication and bcrypt password encryption are implemented to protect user information.
- **Performance Efficiency:** RESTful APIs and optimized frontend components provide faster response and smooth user interaction.

- **Usability:** The system provides a simple and responsive interface that can be easily used by individuals.

3.1.4. Feasibility Analysis

Feasibility analysis evaluates whether the proposed system can be successfully developed and implemented using available resources.

3.1.5. Technical Feasibility

This system will be technologically possible since it utilizes modern and popular technology. React.js framework is used in the creation of the frontend, whereas the backend development will be done through the use of Node.js and Express.js. The storage of user and expense information will be done using MongoDB.

3.1.6. Operational Feasibility

The system provides a simple and user-friendly interface that allows users to manage expenses without requiring advanced technical knowledge. Users can easily register, log in, and perform expense management operations through the dashboard.

3.1.7. Economic Feasibility

The project is economically feasible because it uses open-source technologies such as React.js, Node.js, Express.js, and MongoDB. The main resources required are development time, software tools, and testing environments, resulting in low implementation costs.

3.1.8. Schedule Feasibility

The project was completed within the academic semester according to the planned development phases, including requirement analysis, design, implementation, testing, and documentation.

Table 3.1: Project Schedule

Phases	Weeks										
	1	2	3	4	5	6	7	8	9	10	11
Requirement Gathering	■										
System Design		■	■								
Backend Development			■	■	■	■					
Frontend Development				■	■	■	■				
Integration and Testing								■	■		
Deployment & Documentation										■	■

3.2. Analysis

3.2.1. Data Flow Diagram (DFD)

This Data Flow Diagram at the context level captures the interface between the User and the Expense Tracker Application. The major external entity is the User who uses the system through the provision of registration details, login details, and expenses.

The User provides authentication and expense operations requests which are processed by the system, interacting with the MongoDB database and giving the necessary information regarding logins, expenses, and other operations. The system controls user authentication, storage, search, updates, and deletions of user's expenses.

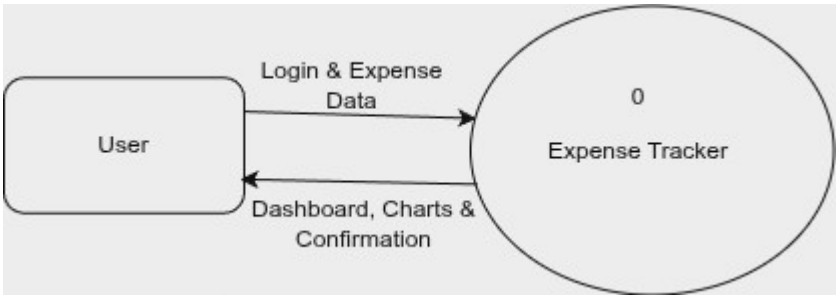

Figure 3.2: Context Diagram of Expense Tracker

4. SYSTEM DESIGN

4.1. Forms and Report Design

Forms are used to collect and display information required for system operations. In the Expense Tracker, forms are designed to provide simple interaction between users and the system. The major forms included in the application are:

- **Registration Form:** Allows new users to create an account by providing personal details such as name, email, and password.
- **Login Form:** Allows registered users to securely access their accounts.
- **Add Expense Form:** Enables users to enter expense details including title, amount, category, payment method, date, and description.
- **Edit Expense Form:** Allows users to update existing expense information.
- **Dashboard View:** Displays summarized expense information and available expense records.

Reports provide information about a collection of records. In the Expense Tracker, the expense list acts as a report where users can view their recorded expenses along with details such as category, amount, and date.

4.2. Interface and Dialogue Design

The user interface of the Expense Tracker is intended to be simple, responsive, and easy-to-use. Users are given the login and register options as soon as they open the application. New users can create their account while existing users can log in.

Upon authentication, users are navigated to the dashboard where they can perform actions on their expenses. Options like adding new expenses, viewing expenses history, editing existing data, and deleting unimportant data are all available on the dashboard.

The user interface of the expenses management module consists of forms for information input and tables/cards for displaying expense data.

4.3. Deployment Diagram

Deployment diagram depicts the physical layout of the software and its interaction process. In Expense Tracker, the React application front end communicates with the back-end server using RESTful APIs.

Frontend of the application is built using React.js framework and runs via a web browser. Backend is coded in Node.js and Express.js framework and performs functionalities like authentication, business logic, and API request processing. The backend connects with MongoDB for storing and retrieving the data of users and expenses.

JWT based authentication ensures the secure exchange of information between the client and server. Environment variables are used for configuration of sensitive information like database connection and authentication credentials.

The overall deployment structure consists of:

- **Client Layer:** React.js application accessed through a web browser.
- **Server Layer:** Node.js and Express.js API server handling application logic.
- **Database Layer:** MongoDB database storing user and expense records.

Figure 4.1: Deployment Diagram of Expense Tracker

5. IMPLEMENTATION AND TESTING

5.1. Implementation

The implementation phase focused on developing the Expense Tracker system based on the system design and requirements identified in previous chapters. The application was developed using the MERN Stack, including MongoDB, Express.js, React.js, and Node.js. Various development tools were used for coding, testing, and version management.

5.1.1. Tools Used

Draw.io:
Used for designing system diagrams such as use case diagrams, context diagrams, and deployment diagrams to visualize system structure and workflow.

Git & GitHub:
Used for version control, source code management, and maintaining project backups.

Postman:
Used for testing RESTful APIs and verifying backend functionality before frontend integration.

Visual Studio Code:
Used as the primary development environment for both frontend and backend development.
It provided useful extensions and tools for efficient coding and debugging.

JavaScript:
Used for both frontend and backend development.

React.js:
Used for creating interactive and responsive user interfaces with reusable components.

Node.js and Express.js:
Used for developing the backend server and implementing RESTful APIs.

MongoDB:
MongoDB was used to store user information and expense records. Its flexible document-based structure allows efficient storage and management of different expense details such as category, amount, payment method, and date.

5.2. Implementation Details of Modules

5.2.1. User Authentication Module

The authentication module provides secure user registration and login functionality. User passwords are encrypted using bcrypt before storage, and JWT authentication is used to verify user identity. Protected routes ensure that only authenticated users can access their expense records.

5.2.2. Expense Management Module

This is the main module of the system that allows users to manage their expenses. Users can:

- Add new expenses
- View expense history
- Update existing expenses
- Delete unwanted expense records

Each expense record is linked with the respective user account to maintain privacy.

5.2.3. Dashboard Module

The dashboard provides users with an overview of their expense records. It displays available expenses and provides navigation options for adding, editing, and managing records.

5.2.4. API Module

RESTful APIs were developed using Express.js to enable communication between the frontend and backend. These APIs handle authentication requests, expense operations, and database interactions.

5.3. Testing

After completing the development phase, testing was performed to identify and fix errors before final deployment. The main objective of testing was to verify that all modules function correctly and satisfy the system requirements.

Testing was performed using Postman for backend API testing and a web browser for frontend functionality testing.

5.3.1. Test Cases for Unit Testing

Unit testing focuses on testing individual modules separately to ensure that each component performs correctly. The major modules tested were authentication, expense management, and user interface components.

Table 5.1: Authentication Testing

Test Case	Expected Result	Actual Result	Status
User Registration	Account created successfully	Successful	Pass
Duplicate Email Registration	Error message displayed	Successful	Pass
Valid Login	JWT token generated	Successful	Pass
Invalid Login	Access denied	Successful	Pass
Protected Route Access	Authorized user allowed	Successful	Pass

Table 5.2: Expense Module Testing

Test Case	Expected Result	Actual Result	Status
Add Expense	Expense added successfully	Successful	Pass
View Expenses	User expenses displayed	Successful	Pass
Update Expense	Expense updated successfully	Successful	Pass
Delete Expense	Expense removed successfully	Successful	Pass
Unauthorized Access	Access denied	Successful	Pass

5.4. Result Analysis

After implementation and testing, the Expense Tracker system successfully achieved its objectives. All major modules, including authentication, expense management, database operations, and API communication, worked correctly.

The system successfully performed CRUD operations and ensured that users could access only their own expense records. Security mechanisms such as JWT authentication and bcrypt password encryption protected user information from unauthorized access.

The testing results confirmed that the application provided reliable performance, proper error handling, and a user-friendly interface. Therefore, the developed system proved to be an effective and secure solution for managing personal expenses digitally.

6. CONCLUSION AND FUTURE RECOMMENDATIONS

6.1. Expected Outcome

The Expense Tracker project will deliver a functioning web application that will help users manage their personal expenses effectively. Users will be able to create an account securely, log in to their account, and carry out expense operations like creating new expenses, viewing expenses, editing and deleting expenses.

Through the use of JWT and bcrypt technologies, the system provides a safe way of accessing the system and securing user's sensitive financial information. This system will make life easier for users who track expenses using the conventional methods, by offering them a central place where they can store their expenses accurately.

This project showcases how one can build a secure and functional web application using the MERN Stack.

6.2. Future Recommendations

Although the current version of the Expense Tracker successfully fulfills its objectives, several enhancements can be implemented in future versions:

6.2.1. Income Management

Adding income tracking functionality would allow users to monitor both earnings and expenses, helping them calculate savings and financial balance.

6.2.2. Budget Planning

A budgeting feature can be introduced to allow users to set spending limits and receive notifications when expenses exceed their planned budget.

6.2.3. Financial Reports and Analytics

The system can be enhanced with graphical reports, charts, and downloadable PDF/Excel reports to provide better analysis of spending patterns.

6.2.4. Advanced Search and Filtering

Additional filtering options based on date, category, payment method, and amount can improve expense record management.

6.2.5. Mobile Application Development

A mobile application using technologies such as React Native or Flutter can be developed to provide easier access through smartphones.

6.2.6. Cloud Deployment

Deploying the application on cloud platforms would improve accessibility, scalability, and availability for users.

6.2.7. AI-Based Expense Analysis

Artificial Intelligence can be integrated to analyze spending behavior, predict future expenses, and provide personalized financial recommendations.

REFERENCES

1. Bodie, Z., Kane, A., & Marcus, A. J. (2021). Personal Finance (2nd ed.). McGraw-Hill Education.
2. Kapoor, J. R., Dlabay, L. R., & Hughes, R. J. (2020). Personal Finance (14th ed.). McGraw-Hill Education.
3. Gitman, L. J., Joehnk, M. D., & Billingsley, R. S. (2022). Personal Financial Planning (15th ed.). Cengage Learning.
4. OECD. (2020). OECD/INFE 2020 International Survey of Adult Financial Literacy. OECD Publishing. Available: https://www.oecd.org/
5. Investopedia. (2024). Expense Tracking: Definition, Benefits, and Best Practices. Available: https://www.investopedia.com/
6. Consumer Financial Protection Bureau. (2023). Budgeting and Saving Resources. Available: https://www.consumerfinance.gov/